

Aplicaciones y Periféricos del Microcontrolador ATmega328P

1st Candela Vargas

5.645.897-6

Mercedes

2nd Neyzer Marcenal

5.529.314-9

Fray Bentos

3rd Julia Meléndrez

5.397.666-4

Fray Bentos

candela.vargas@estudiantes.utec.edu.uy neyzer.marcenal@estudiantes.utec.edu.uy julia.melendrez@estudiantes.utec.edu.uy

Abstract—En este laboratorio se desarrollaron diferentes aplicaciones utilizando el microcontrolador ATmega328P programado en lenguaje ensamblador AVR, con el objetivo de aprender y aplicar diferentes funciones y recursos del microcontrolador. Se implementó el control de una matriz de LEDs, un conversor digital-analógico R-2R, un plotter controlado mediante PLC, comunicación USART/UART entre microcontroladores y un sistema de control para una puerta de garaje. Para el desarrollo de estas aplicaciones se utilizaron recursos como temporizadores, interrupciones, tablas de consulta (LUT), comunicación serial y manejo de la pila. La práctica permitió integrar conceptos de programación y electrónica, aplicándolos al control y comunicación entre diferentes dispositivos dentro de sistemas embebidos.

Index Terms—ATmega328P, AVR, lenguaje ensamblador, USART, matriz de LEDs, DAC R-2R, PLC, sistemas embebidos.

I. INTRODUCCIÓN

En los sistemas mecatrónicos, los microcontroladores cumplen un papel fundamental en la adquisición de información, el procesamiento de datos y el control de diferentes dispositivos. A medida que aumenta la complejidad de una aplicación, resulta necesario incorporar distintos recursos del microcontrolador que permitan gestionar tareas de comunicación, temporización y control de manera eficiente.

En este segundo laboratorio de la unidad curricular Tecnologías de Microprocesamiento se trabajó con el microcontrolador ATmega328P, profundizando su programación en lenguaje ensamblador e incorporando diferentes periféricos y recursos internos. Entre ellos se encuentran los temporizadores, las interrupciones, la comunicación USART/UART, el uso de tablas de consulta (LUT) y el manejo de la memoria mediante el puntero de pila.

A lo largo de la práctica se desarrollaron diferentes aplicaciones que permitieron integrar estos conceptos, incluyendo el control de una matriz de LEDs, la generación de señales mediante un conversor DAC R-2R, el control de un plotter mediante la comunicación entre el ATmega328P y un PLC, la transmisión de datos entre microcontroladores y la implementación de un sistema de control para una puerta de garaje. Estas actividades permitieron aplicar de forma práctica los conocimientos adquiridos y comprender la interacción entre software y hardware dentro de un sistema embebido.

II. OBJETIVOS

A. Objetivo general

Desarrollar e implementar diferentes sistemas utilizando el microcontrolador ATmega328P programado en lenguaje ensamblador, integrando sus periféricos y recursos internos para el control, la comunicación y el procesamiento de señales en distintas aplicaciones.

B. Objetivos específicos

- Implementar el control de distintos dispositivos, como una matriz de LEDs, un conversor DAC R-2R y un plotter, mediante la programación del ATmega328P y la adecuada gestión de sus entradas y salidas.
- Aplicar recursos del microcontrolador como temporizadores, interrupciones, tablas de consulta (LUT) y comunicación USART/UART para controlar el funcionamiento y la sincronización de las diferentes aplicaciones desarrolladas.
- Integrar el microcontrolador con diferentes dispositivos y sistemas externos, estableciendo comunicación entre microcontroladores y realizando la interfaz con un PLC para el control del plotter.

III. MATERIALES UTILIZADOS

- Arduino Uno con microcontrolador ATmega328P.
- Protoboard.
- LEDs.
- Resistencias.
- Pulsadores.
- DIP-switches
- Cables de conexión.
- Cable USB para programación y alimentación.
- Plotter

IV. SOFTWARE UTILIZADO

- Microchip Studio 7.
- PICSIMLAB.
- Proteus 8 Professional.
- AVR Assembler.
- Git y GitHub.

V. MARCO TEÓRICO

A. Matriz de LEDs y multiplexación

Una matriz de LEDs consiste en un conjunto de diodos emisores de luz organizados en filas y columnas, donde cada LED se encuentra ubicado en la intersección entre una fila y una columna. Esta disposición permite controlar una gran cantidad de LEDs utilizando un número reducido de conexiones. Por ejemplo, una matriz de 8×8 contiene 64 LEDs, pero requiere únicamente ocho conexiones correspondientes a las filas y ocho a las columnas.

Para controlar individualmente los LEDs se emplea la multiplexación, técnica que consiste en activar secuencialmente una fila de la matriz mientras se establece en las columnas el patrón correspondiente a los LEDs que se desean encender. Este procedimiento se repite rápidamente para cada una de las filas. Debido a la velocidad de refresco y a la persistencia de la visión humana, la secuencia es percibida como una imagen continua en lugar de una sucesión de filas encendiéndose y apagándose.

Mediante la modificación de los patrones aplicados durante este barrido es posible representar caracteres, símbolos e imágenes. Asimismo, desplazando progresivamente dichos patrones se pueden generar mensajes en movimiento, como el requerido en esta práctica. La correcta activación de filas y columnas depende de la configuración eléctrica de la matriz, por lo que debe considerarse su polaridad y distribución de terminales.

B. Comunicación USART/UART

La comunicación UART (Universal Asynchronous Receiver-Transmitter) permite realizar una transmisión serial de información, enviando los datos bit a bit entre dos dispositivos. Al tratarse de una comunicación asíncrona, no utiliza una señal de reloj compartida, por lo que el transmisor y el receptor deben configurarse previamente con la misma velocidad de transmisión o tasa de baudios.

La transmisión se realiza principalmente mediante las líneas TX (transmisión) y RX (recepción). Para comunicar dos dispositivos, la salida TX de uno se conecta a la entrada RX del otro y viceversa. En una transmisión UART, cada dato se organiza en una trama que incluye un bit de inicio (start bit), los bits de datos, un bit de paridad opcional y uno o más bits de parada (stop bits).

Por otra parte, USART (Universal Synchronous/Asynchronous Receiver-Transmitter) puede operar tanto en modo asíncrono como síncrono. En modo asíncrono presenta un funcionamiento equivalente a UART, mientras que en modo síncrono incorpora una señal de reloj para sincronizar la transmisión.

En el ATmega328P se dispone de un periférico USART configurable para realizar este tipo de comunicación. En el presente laboratorio se emplea en modo asíncrono, permitiendo el intercambio de información entre el microcontrolador y otros dispositivos del sistema.

C. Tabla de consulta (LUT)

Una tabla de consulta o LUT (Look-Up Table) es una estructura que almacena un conjunto de valores previamente definidos o calculados. Su utilización permite obtener directamente un valor almacenado mediante una posición de la tabla, evitando realizar determinados cálculos durante la ejecución del programa.

En sistemas con microcontroladores, las LUT pueden utilizarse para almacenar secuencias de datos que posteriormente son leídas de manera ordenada para generar patrones o señales específicas. De esta forma, el microcontrolador accede sucesivamente a los valores almacenados y los utiliza como datos de salida.

En este laboratorio, la LUT se emplea para almacenar los valores correspondientes a las muestras de una señal. Estos valores son enviados secuencialmente desde el ATmega328P hacia un convertor digital-analógico (DAC) R-2R, permitiendo reconstruir una señal analógica que posteriormente puede visualizarse mediante un osciloscopio.

D. Convertor digital-analógico DAC R-2R

Un convertor digital-analógico (DAC) R-2R es un circuito que transforma un valor digital binario en un nivel de tensión analógico equivalente. Su estructura está formada por una red repetitiva de resistencias de únicamente dos valores, R y $2R$, dispuestas en forma de escalera.

En esta red, cada bit de la entrada digital posee un determinado peso binario, por lo que la combinación de los bits establece el nivel de tensión obtenido a la salida. Una de las ventajas de esta configuración es que requiere solamente dos valores de resistencia, simplificando su implementación frente a otras redes de resistencias ponderadas.

La cantidad de niveles analógicos que pueden representarse depende del número de bits del convertor. Para un DAC de n bits existen 2^n niveles posibles y su resolución ideal puede expresarse como:

$$\text{Resolución} = \frac{V_{\text{ref}}}{2^n} \quad (1)$$

En el presente laboratorio se utiliza un DAC R-2R de 8 bits, por lo que pueden representarse $2^8 = 256$ niveles digitales. El ATmega328P envía secuencialmente al convertor los valores almacenados en una tabla de consulta (LUT), obteniendo diferentes niveles de tensión que permiten conformar una señal analógica y visualizarla mediante un osciloscopio.

E. Temporizadores y contadores

Los temporizadores y contadores (*timers/counters*) son periféricos internos del microcontrolador que permiten medir intervalos de tiempo, generar retardos precisos y contabilizar eventos. Su funcionamiento se basa en un registro cuyo valor se incrementa o decrementa a partir de una determinada fuente de pulsos.

Cuando los pulsos provienen del reloj interno del microcontrolador, el periférico funciona como temporizador, permitiendo establecer intervalos de tiempo de manera precisa. En

cambio, cuando los pulsos provienen de una fuente externa, puede utilizarse como contador de eventos.

La frecuencia con la que se incrementa el temporizador puede modificarse mediante un preescalador (*prescaler*), que divide la frecuencia del reloj antes de aplicarla al contador. Esto permite adaptar el rango de temporización a las necesidades de la aplicación.

En el ATmega328P se dispone de temporizadores de 8 y 16 bits, cuya capacidad determina el rango de valores que pueden alcanzar. En este laboratorio, los temporizadores permiten controlar parámetros temporales del sistema, como la velocidad de desplazamiento de la matriz de LEDs y la frecuencia de la señal generada mediante el DAC R-2R.

F. Interrupciones

Las interrupciones son mecanismos que permiten al microcontrolador responder ante determinados eventos sin necesidad de comprobarlos continuamente dentro del programa principal. Cuando ocurre un evento habilitado como interrupción, la CPU interrumpe temporalmente la ejecución normal y ejecuta una rutina específica denominada *Interrupt Service Routine (ISR)*.

Una vez finalizada la rutina de servicio, el microcontrolador puede continuar con la ejecución del programa desde el punto en el que fue interrumpido. De esta manera, las interrupciones permiten responder rápidamente ante eventos que requieren atención inmediata.

Las interrupciones pueden originarse por eventos externos, como el cambio de estado de un pin conectado a un pulsador o sensor, o por eventos internos generados por periféricos del microcontrolador, como temporizadores o módulos de comunicación serial. Para su utilización deben configurarse las fuentes de interrupción correspondientes y habilitarse tanto la interrupción específica como el sistema global de interrupciones.

En el presente laboratorio, este mecanismo se aplica al sistema de control de una puerta de garaje. El sensor de obstáculos debe generar una interrupción que permita detener inmediatamente el movimiento del motor y desactivar la alarma, priorizando este evento frente a la ejecución normal del sistema.

G. Puntero de pila

La pila (*stack*) es una zona de memoria utilizada para almacenar información de manera temporal siguiendo una estructura LIFO (*Last In, First Out*), donde el último dato almacenado es el primero en ser recuperado.

El puntero de pila o *Stack Pointer (SP)* es un registro que mantiene la dirección correspondiente a la parte superior de la pila. En el ATmega328P, la pila se encuentra en la memoria SRAM y crece desde direcciones de memoria superiores hacia direcciones inferiores.

El acceso a la pila puede realizarse mediante instrucciones como *PUSH* y *POP*. La instrucción *PUSH* permite almacenar temporalmente el contenido de un registro en la pila, mientras que *POP* permite recuperar posteriormente dicho valor, manteniendo el principio LIFO.

La pila también cumple un papel importante durante las llamadas a subrutinas y la ejecución de interrupciones, ya que permite conservar información necesaria para posteriormente continuar con la ejecución normal del programa. Por este motivo, al programar el ATmega328P en lenguaje ensamblador es necesario inicializar correctamente el puntero de pila antes de utilizar estas operaciones.

H. Plotter monocromático y sistema de control

Un plotter monocromático didáctico es un sistema mecatrónico que permite realizar trazados mediante el movimiento controlado de un elemento de escritura sobre una superficie. Este tipo de sistema permite integrar conceptos de control de movimiento, automatización, accionamiento de actuadores y gestión de entradas y salidas.

El movimiento del plotter puede describirse mediante desplazamientos en dos ejes, permitiendo posicionar el elemento de trazado dentro de su área de trabajo. Para realizar una figura es necesario coordinar la dirección y el tiempo de cada desplazamiento con la activación o desactivación del mecanismo encargado de efectuar el trazo.

En el sistema utilizado en el laboratorio, el control del plotter se realiza mediante un controlador lógico programable (PLC), encargado de recibir las señales de control y accionar los mecanismos correspondientes. El sistema dispone de movimientos hacia arriba, abajo, izquierda y derecha, además de una válvula neumática que permite subir o bajar el elemento de trazado.

Para establecer una referencia conocida de posición, el sistema cuenta con una posición de origen o *home*. Al iniciar o reiniciar el equipo, el mecanismo debe desplazarse hasta la esquina superior derecha, estableciendo dicha posición como referencia para los movimientos posteriores. Además, se dispone de una parada de emergencia (*E-STOP*) que permite detener la secuencia de movimiento ante una situación que requiera la interrupción inmediata del proceso.

I. Interfaz mediante relés y MOSFET

En sistemas de control es frecuente utilizar elementos de conmutación para establecer una interfaz entre dispositivos que trabajan con diferentes características eléctricas. Entre las alternativas más utilizadas se encuentran los relés electromecánicos y los transistores MOSFET.

Un relé electromecánico funciona como un interruptor accionado mediante una bobina. Al energizarse, sus contactos cambian de estado, permitiendo controlar otro circuito sin establecer una conexión eléctrica directa entre la señal de control y el circuito conmutado. Esta característica permite proporcionar aislamiento eléctrico entre ambas etapas. Como contrapartida, al poseer elementos mecánicos, su velocidad de conmutación es menor y sus contactos presentan desgaste con el uso.

Por otro lado, un MOSFET es un dispositivo semiconductor que permite realizar la conmutación de una señal de manera electrónica. Al no poseer partes móviles, presenta una mayor velocidad de conmutación y no sufre desgaste mecánico,

por lo que resulta adecuado para aplicaciones que requieren accionamientos frecuentes.

En el presente laboratorio, los relés electromecánicos o MOSFET se utilizan como interfaz entre el ATmega328P y el PLC encargado del control del plotter. El microcontrolador genera las señales digitales correspondientes a los movimientos y al accionamiento del elemento de trazado, mientras que la etapa de interfaz permite transmitir dichas órdenes hacia las entradas del PLC.

VI. METODOLOGÍA

A. Control de matriz de LEDs

Para el desarrollo de la tarea se utilizó un microcontrolador ATmega328P para controlar una matriz LED de 8×8 mediante comunicación USART. El programa fue desarrollado en lenguaje ensamblador, configurando los puertos del microcontrolador necesarios para el control de las filas y columnas de la matriz, además de los pines destinados a la comunicación serial.

El programa se diseñó para permitir la selección de tres opciones de visualización mediante el envío de caracteres desde el monitor serial. La opción 1 corresponde al desplazamiento del mensaje “VAMOS MARCELOOOO”, mientras que las opciones 2 y 3 permiten visualizar los dibujos “UWU” y “XD”, respectivamente.

Para controlar la matriz LED se utilizó multiplexación, activando sucesivamente cada una de sus filas y estableciendo en las columnas el patrón correspondiente. Este proceso se realiza de manera continua y a suficiente velocidad para que las diferentes filas sean percibidas visualmente como una única imagen.

Una vez desarrollado el programa, se realizó la simulación del sistema en Proteus. En esta etapa se verificó la comunicación USART, la selección de las diferentes opciones y la correcta representación del mensaje y los dibujos en la matriz LED.

Posteriormente, se realizó el montaje físico utilizando un Arduino Uno con microcontrolador ATmega328P, una matriz LED de 8×8 , resistencias limitadoras de corriente, protoboard y cables de conexión. El programa fue compilado y cargado en el microcontrolador y se realizaron las pruebas utilizando el monitor serial configurado a 9600 baudios.

Finalmente, se probaron las tres opciones programadas y se comparó el comportamiento del montaje físico con el obtenido previamente en Proteus, realizando los ajustes necesarios hasta obtener el funcionamiento esperado del sistema.

B. Conversor DAC R-2R

Para el desarrollo de esta parte se utilizó el microcontrolador ATmega328P programado en lenguaje ensamblador. El objetivo fue generar las señales 3 y 11 asignadas al grupo mediante un conversor digital-analógico R-2R de 8 bits.

Los valores correspondientes a cada señal se almacenaron en tablas de consulta o LUT (*Look-Up Table*) dentro de la memoria de programa. Cada tabla contiene 256 muestras de 8

bits que son leídas de forma secuencial y enviadas hacia las salidas digitales conectadas al DAC R-2R.

La señal 3 está formada por valores que aumentan progresivamente desde 0×00 hasta $0 \times FF$, mientras que la señal 11 utiliza una secuencia de valores crecientes y decrecientes. Al tratarse de señales periódicas, una vez finalizado el recorrido de la tabla se vuelve a comenzar desde la primera muestra.

Además, se incorporó comunicación USART para permitir la selección de la señal desde una computadora y Timer1 para controlar el tiempo entre muestras y, por lo tanto, modificar la frecuencia de salida.

1) *Salidas digitales y DAC R-2R*: Para completar los ocho bits necesarios para el conversor se utilizaron seis salidas del puerto B y dos del puerto C. Los bits 0 a 5 de cada muestra se envían mediante PB0–PB5 y los bits 6 y 7 mediante PC0 y PC1.

La red R-2R se construyó utilizando resistencias de $10 \text{ k}\Omega$ para R y $20 \text{ k}\Omega$ para 2R. De esta manera, las ocho salidas digitales del microcontrolador forman un único nivel de tensión analógico en la salida de la red.

En el programa se implementó la rutina DAC_SALIDA. En ella, una máscara $0 \times 3F$ permite conservar los seis bits inferiores de la muestra para enviarlos al puerto B. Los dos bits restantes se desplazan mediante instrucciones LSR y son enviados al puerto C.

2) *Lectura de las LUT*: Las muestras de ambas señales se almacenaron en memoria Flash mediante la directiva .DB. Para acceder a estos valores se utilizó el puntero Z, formado por los registros ZH y ZL.

Al seleccionar una señal, el puntero se carga con la dirección inicial de la LUT correspondiente. La lectura de cada muestra se realiza mediante:

LPM DATO, Z+

La instrucción LPM lee un byte almacenado en memoria de programa y lo guarda en el registro DATO. El uso de Z+ permite que el puntero avance automáticamente hacia la siguiente posición de la tabla.

Se utilizó además un registro como contador de muestras. Luego de recorrer las 256 posiciones, el programa vuelve al comienzo de la tabla y continúa con la generación de la señal.

3) *Control de frecuencia mediante Timer1*: Para controlar el tiempo entre muestras se utilizó Timer1 en modo CTC (*Clear Timer on Compare Match*) con un prescaler de 64. Con una frecuencia de reloj de 16 MHz se utilizaron tres valores diferentes en el registro OCR1A para obtener tres velocidades de funcionamiento.

TABLE I
CONFIGURACIONES DE VELOCIDAD UTILIZADAS

Modo	OCR1A	Tiempo por muestra
Lento	499	2 ms
Medio	249	1 ms
Rápido	124	0.5 ms

La rutina ESPERAR_TIMER consulta la bandera OCF1A. Cuando el contador alcanza el valor definido en OCR1A, la

bandera se limpia y el programa continúa con la siguiente muestra.

Por lo tanto, al disminuir OCR1A se reduce el tiempo de espera entre muestras, mientras que al aumentarlo se genera una espera mayor.

4) *Comunicación USART*: El módulo USART se configuró a 9600 baudios en formato 8N1. Para una frecuencia de reloj de 16 MHz se utilizó el valor 103 en UBRR0.

Al iniciar el programa se transmite el siguiente menú:

```
DAC R-2R
1 - Señal 11
2 - Señal 3
+ - Mas rapido
- - Mas lento
M - Menu
```

Los caracteres recibidos son comparados dentro de la rutina de recepción. La opción '1' selecciona la señal 11 y '2' la señal 3. Los comandos '+' y '-' modifican el nivel de velocidad, mientras que 'M' permite volver a transmitir el menú.

La recepción USART también se consulta mientras se está recorriendo una LUT. Esto permite cambiar de señal o de velocidad durante la ejecución sin tener que reiniciar el microcontrolador.

5) *Flujo del programa*: El funcionamiento general del programa se representa mediante el diagrama de flujo de la Fig. 1. Primero se configuran el Stack Pointer, los puertos digitales, USART y Timer1. Luego se comprueba qué señal se encuentra seleccionada y se carga en el puntero Z la dirección de su LUT.

A partir de ese momento se lee una muestra, se envía hacia el DAC R-2R y se espera el tiempo definido por Timer1. Durante el recorrido se comprueba también si existe un nuevo comando recibido mediante USART. Una vez recorridas las 256 muestras se vuelve al comienzo de la tabla.

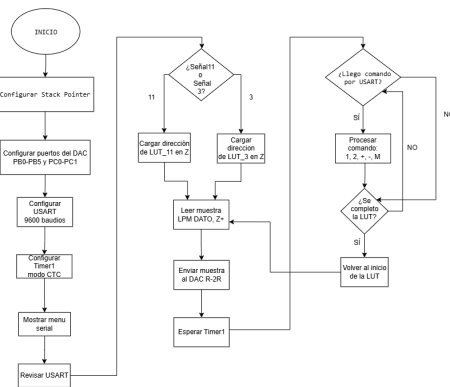


Fig. 1. Diagrama de flujo del generador de señales mediante LUT y DAC R-2R.

6) *Implementación física y pruebas*: Una vez finalizado el programa se compiló en Microchip Studio y se cargó el archivo .hex generado en un Arduino Uno con microcontrolador ATmega328P.

La red R-2R fue montada en protoboard y conectada a las ocho salidas digitales utilizadas por el programa. La salida analógica del conversor se conectó a un osciloscopio, utilizando una referencia de GND común entre el Arduino, la protoboard y el instrumento.

Para realizar las pruebas se utilizó también el monitor serial conectado por USB y configurado a 9600 baudios. El procedimiento de prueba consistió en seleccionar ambas señales y observar su salida en el osciloscopio en los tres modos de velocidad disponibles.

También se realizó la simulación del sistema en Proteus, utilizando el archivo .hex generado, la red R-2R, una terminal serial y un osciloscopio virtual para comprobar la generación de las señales y el funcionamiento de la comunicación USART.

7) *Versionado del programa*: El desarrollo del código se gestionó mediante Git y GitHub. Las funciones principales se incorporaron en diferentes etapas para mantener un registro del avance realizado durante la implementación.

TABLE II
PRINCIPALES ETAPAS DE DESARROLLO REGISTRADAS MEDIANTE GIT

Commit	Modificación realizada
1	Configuración inicial del DAC y USART
2	Incorporación de la señal 11 y Timer1
3	Incorporación de la señal 3 y menú serial
4	Ajuste de frecuencia mediante Timer1

El código fuente completo y su historial de modificaciones se encuentran disponibles en el repositorio de GitHub indicado en el Apéndice.

C. Control del plotter

1) *Interfaz de Hardware y Periféricos*: La interfaz entre el microcontrolador ATmega328P y el PLC del plotter se configuró utilizando el puerto D (PORTD):

- **PD2 / PD3**: Pulsos de control para bajar (X0) y subir (X1) la pluma neumática[cite: 1].
- **PD4 / PD5**: Movimiento en el eje Y (Abajo X5 / Arriba X6).
- **PD6 / PD7**: Movimiento en el eje X (Izquierda X7 / Derecha X10).

La dirección de datos se inicializó definiendo DDRD = 0b11111100 (0xFC), lo que mantiene las salidas en estado bajo al energizar el sistema.

El periférico USART se configuró a 9600 baudios (UBRR0L = 103 para $f_{osc} = 16\text{ MHz}$) en formato 8N1 mediante el registro UCSR0C. La subrutina MOSTRAR_MENU lee la cadena de texto desde la memoria Flash (MENU<<1) empleando la instrucción lpm r16, Z+ y la transmite por el puerto serie. Durante la recepción (USART_RX), una estructura condicional basada en cpi y breq evalúa el carácter ingresado para derivar la ejecución a la rutina correspondiente ('1' a '4', 'P', 'T').

2) Generación de Movimiento y Figuras Básicas:

- **Control de Pluma**: Las subrutinas lapiz_abajo y lapiz_arriba generan un pulso en el pin correspondiente (sbi / cbi), desacoplado mecánicamente mediante

la rutina `delay_rele` (bucles anidados en `r18`, `r19` y `r20`) para asegurar la conmutación física del actuador.

- **Control de Ejes:** Los desplazamientos continuos se ejecutan mediante subrutinas parametrizadas en el registro `r21` (`pasos_abajo`, `pasos_derecha`, etc.) y rutinas de interpolación diagonal (`diagonal_abajo_derecha`, etc.) asistidas por el retardo corto `delay_paso`.
- **Trazado de Figuras:** El triángulo y la figura libre (corazón) se construyen combinando segmentos rectos y diagonales[cite: 1]; el círculo se aproxima mediante un polígono regular de 12 tramos interpolados; y el pentagrama utiliza bucles anidados con relaciones de paso 1 : 2 y 1 : 3.

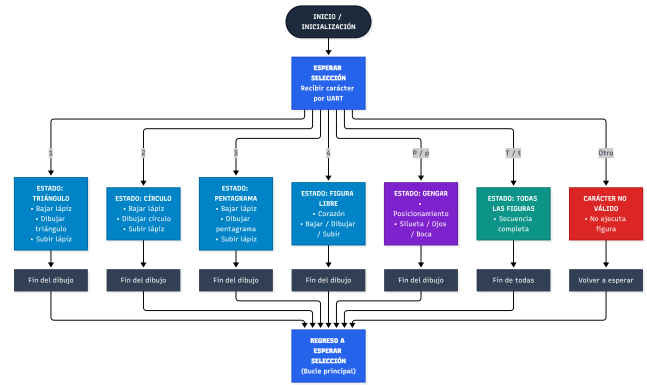


Fig. 2. Diagrama de estados de la FSM para el control del Plotter.

3) *Implementación de Gengar (LUT e Inversión Cinemática)*: El trazado del Pokémon asignado (Gengar) se estructuró en cuatro etapas principales:

- 1) **Posicionamiento:** El desplazamiento inicial de 260 pasos en el eje X se dividió en dos llamadas consecutivas de 130 pasos para prevenir el desbordamiento del registro de 8 bits (`r21`).
- 2) **Lectura de LUT:** El contorno exterior se almacenó en memoria Flash (`GENGAR_MATRIZ`) en formato de pares (Dirección, Cantidad de pasos). La rutina `gengar_leer_matriz` lee secuencialmente estos valores con la instrucción `lpm` hasta detectar la constante de fin `G_FIN` (0).
- 3) **Inversión Cinemática:** Debido a la disposición física del eje X en el laboratorio, la rutina `gengar_ejecutar_movimiento` realiza un remapeo por software: las direcciones lógicas hacia la derecha ejecutan físicamente `pasos_izquierda` y viceversa.
- 4) **Rasgos Internos:** Concluido el contorno, se eleva la pluma (`lapiz_arriba`), se aplica un desplazamiento de posición y se dibujan de forma independiente las órbitas oculares y la boca con tres dientes verticales.

4) *Secuencia Completa (Opción 'T')*: La selección de la opción 'T' invoca la subrutina `dibujar_todas`, la cual ejecuta consecutivamente cada rutina de dibujo e intercala desplazamientos espaciales con la pluma elevada para distribuir las figuras de manera ordenada a lo largo del área de trabajo.

D. Comunicación USART - UART

El desarrollo del sistema de comunicación unidireccional comprende la interconexión de dos microcontroladores AT-mega328P operando a una frecuencia de reloj de 16 MHz. El microcontrolador emisor captura una entrada binaria manual de 3 bits y la transmite de forma serie hacia el microcontrolador receptor, el cual decodifica la información recibida para activar una única salida discreta dentro de un grupo de 8 LEDs.

1) *Configuración de la Comunicación USART*: Para establecer el enlace punto a punto entre la línea de transmisión TXD (pin PD1) del emisor y la línea de recepción RXD (pin PD0) del receptor, se configuró el periférico USART a una tasa de transferencia de 9600 bps en modo asíncrono normal. El valor cargado en el registro `UBRR0` (`UBRR0H:UBRR0L`) se determinó mediante la siguiente ecuación:

$$UBRR0 = \frac{f_{osc}}{16 \times \text{Baud}} - 1 = \frac{16\,000\,000}{16 \times 9600} - 1 = 103.16 \approx 103 \quad (2)$$

Ambos microcontroladores fueron programados con un formato de trama de 8 bits de datos, 1 bit de parada y sin paridad (8N1) mediante el registro `UCSR0C`:

- **Emisor:** Se habilitó únicamente la transmisión ajustando el bit `TXEN0` en el registro `UCSR0B`.
- **Receptor:** Se habilitó únicamente la recepción ajustando el bit `RXEN0` en el registro `UCSR0B`.

2) *Implementación del Microcontrolador Emisor*: El subsistema transmisor gestiona la lectura de los pines PD2, PD3 y PD4 del puerto D. La secuencia de ejecución implementada en ensamblador consta de los siguientes pasos:

- 1) **Inicialización e I/O:** Se configuran los pines PD2..PD4 como entradas digitales sin resistencia *pull-up* interna, asumiendo una red de resistencias *pull-down* externas en el circuito físico.
- 2) **Acondicionamiento de Datos:** Mediante la instrucción `in r16, PIND`, se lee el estado lógico de las entradas. Se ejecutan dos desplazamientos lógicos hacia la derecha (`lsl r16`) para alinear los pines PD2..PD4 a los bits menos significativos (posiciones 0, 1 y 2).

- 3) **Enmascaramiento:** Se aplica una máscara AND inmediata (`andi r16, 0x07`) para aislar el valor numérico comprendido entre 0 y 7, garantizando la limpieza de los bits superiores.
- 4) **Transmisión:** Se sondea la bandera del registro de datos libre (UDRE0) en el registro UCSR0A mediante un bucle de espera. Al estar disponible el *buffer*, el dato de 3 bits se carga en el registro UDR0 para su envío.

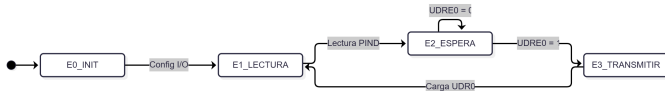


Fig. 3. Diagrama de la Máquina de Estados Finitos del Emisor (FSM-TX).

3) **Implementación del Microcontrolador Receptor y Decodificador:** El subsistema receptor adquiere el dato serie y realiza una conversión de binario a decodificación 1 de 8 mediante procesamiento en *software*:

- 1) **Configuración de Salidas:** Se configuran los pines PC0..PC5 (puerto C) y PB0..PB1 (puerto B) como salidas digitales (DDRC = 0x3F, DDRB = 0x03). Todos los pines se inicializan en nivel bajo.
- 2) **Recepción:** El flujo de programa entra en un bucle de espera activa sondeando la bandera de recepción completa RXC0 en UCSR0A. Tras la recepción de un byte, este es extraído de UDR0 y filtrado nuevamente con la máscara 0x07.
- 3) **Algoritmo Decodificador 3 a 8:** Se inicializa un registro auxiliar con el valor base 1 (`ldi r17, 1`), representando una máscara tipo *one-hot* (00000001b). Un contador iterativo decreuenta el valor recibido mientras aplica desplazamientos lógicos a la izquierda (`lsl r17`), posicionando el bit '1' en la casilla requerida (2^N , donde $N \in [0, 7]$).
- 4) **Mapeo Físico de Salidas:** Debido a la restricción de pines contiguos en el encapsulado, la salida decodificada se segmenta entre dos puertos:
 - **Caso $N \in [0, 5]$ (enviar_portc):** Se limpia el puerto B (`PORTB = 0`) para apagar cualquier LED previo y se escribe directamente la máscara `r17` sobre `PORTC` (asociado a las salidas A0 a A5).
 - **Caso $N \in [6, 7]$ (enviar_portb):** Se evalúa la condición mediante `cpi r16, 6`. Se limpia el puerto C (`PORTC = 0`). Posteriormente, se desplaza el registro `r17` seis posiciones a la derecha para mapear los bits 6 y 7 a las posiciones PB0 y PB1 (pines D8 y D9).



Fig. 4. Diagrama de la Máquina de Estados Finitos del Receptor (FSM-RX).

E. Sistema de control de puerta de garaje

Para la automatización de la puerta de garaje se utilizó el microcontrolador ATmega328P programado completamente en lenguaje ensamblador. El funcionamiento se organizó mediante una máquina de estados, de forma que el sistema pudiera controlar de manera ordenada la apertura y cierre de la puerta, la lectura de los sensores de posición y la actuación del sistema de seguridad.

Se definieron cinco estados principales: CERRADA, ABRIENDO, ABIERTA, CERRANDO y SEGURIDAD. El sistema comienza en el estado CERRADA. Al detectar la orden de apertura pasa al estado ABRIENDO, donde se activa la salida correspondiente al movimiento de apertura y también la alarma. Cuando se activa el sensor de puerta completamente abierta, el movimiento se detiene y el sistema pasa al estado ABIERTA.

El proceso de cierre funciona de forma equivalente. Desde el estado ABIERTA, la orden de cierre provoca el cambio hacia CERRANDO. Durante este estado se activa la salida correspondiente al sentido contrario del movimiento y se mantiene activa la alarma. Al detectar el sensor de puerta cerrada se desactivan las salidas y el sistema regresa al estado CERRADA.

Se agregó además el estado SEGURIDAD, utilizado cuando se detecta un obstáculo mientras la puerta se encuentra en movimiento. En esta condición las salidas asociadas al movimiento y la alarma son desactivadas inmediatamente. El sistema permanece detenido hasta que el obstáculo desaparece y se recibe una nueva orden de apertura o cierre.

La máquina implementada corresponde al modelo de Moore, ya que las salidas asociadas al movimiento y a la alarma quedan determinadas principalmente por el estado actual del sistema. La Fig. 5 representa los estados definidos y las condiciones utilizadas para realizar las transiciones.

1) **Asignación de entradas y salidas:** Para implementar el sistema se utilizaron principalmente los puertos B y D del ATmega328P. Los pulsadores correspondientes a las órdenes de apertura y cierre y los sensores de posición fueron conectados al puerto D. Por otra parte, el sensor de obstáculos y las salidas correspondientes al movimiento y a la alarma fueron asignados al puerto B.

Las entradas digitales fueron configuradas utilizando las resistencias *pull-up* internas del microcontrolador. Debido a esta configuración, una entrada sin activar presenta un nivel lógico alto, mientras que al presionar un pulsador o activar un sensor el nivel pasa a bajo.

El programa fue desarrollado en lenguaje ensamblador AVR utilizando Microchip Studio. Para representar el estado actual del sistema se utilizó el registro R20, mientras que otros registros se emplearon como auxiliares para la configuración de periféricos, la comunicación USART y el manejo de avisos.

Los diferentes estados fueron definidos mediante constantes numéricas. Dentro del bucle principal, el contenido del registro de estado se compara mediante la instrucción `CPI`, y las instrucciones `BREQ` y `RJMP` permiten dirigir la ejecución hacia el bloque correspondiente. De esta forma, el programa queda

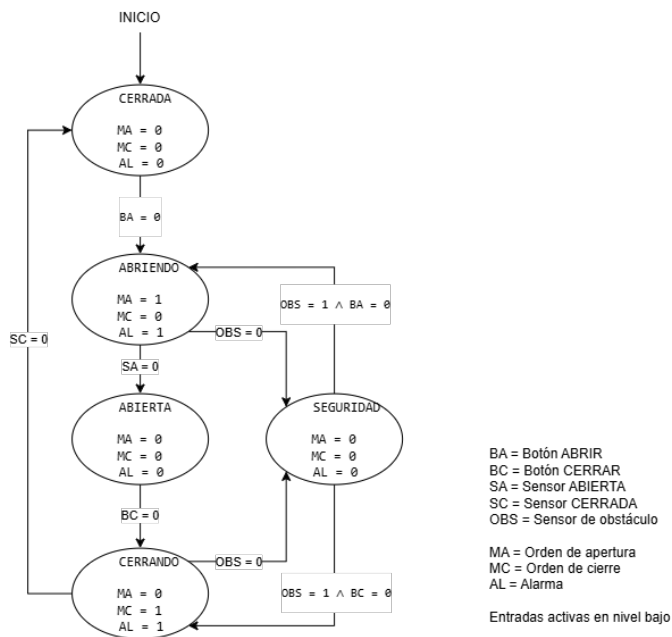


Fig. 5. Máquina de estados tipo Moore utilizada para el control de la puerta de garaje.

dividido en rutinas independientes para los estados CERRADA, ABRIENDO, ABIERTA, CERRANDO y SEGURIDAD.

Las instrucciones SBI y CBI fueron utilizadas para activar y desactivar individualmente las salidas del puerto B. Para la lectura de los pulsadores y sensores se utilizó principalmente la instrucción SBIC. Debido al uso de resistencias *pull-up*, las entradas son consideradas activas cuando presentan un nivel lógico bajo.

También se configuró el *Stack Pointer* al inicio del programa mediante los registros SPH y SPL. Esta configuración es necesaria debido al uso de subrutinas mediante RCALL y RET, además de la rutina de servicio utilizada para la interrupción de seguridad.

2) *Interrupción de seguridad*: Para detectar la presencia de un obstáculo se utilizó el pin PB0, correspondiente a PCINT0. Se eligió una interrupción por cambio de pin debido a que la consigna requería utilizar una fuente de interrupción diferente de INT0 e INT1.

La interrupción se configuró mediante los registros PCICR y PCMSK0. El primero permite habilitar el grupo de interrupciones por cambio correspondiente al puerto B, mientras que el segundo selecciona específicamente PB0 como fuente de interrupción. Finalmente se habilitaron las interrupciones globales mediante la instrucción SEI.

Cuando se produce un cambio en PB0, el microcontrolador ejecuta la rutina ISR_OBSTACULO. Debido a que una interrupción por cambio de pin puede producirse tanto al activar como al liberar el sensor, dentro de la rutina se comprueba nuevamente el nivel lógico de PB0. La acción de seguridad solamente se ejecuta si el sensor se encuentra activo y la puerta estaba en los estados ABRIENDO o CERRANDO.

Al detectarse el obstáculo se desactivan inmediatamente las dos órdenes de movimiento y la alarma, y el estado del sistema cambia a SEGURIDAD. Antes de modificar los registros utilizados en la interrupción se guardan temporalmente en el *Stack* el registro auxiliar y el registro de estado SREG. Antes de regresar al programa principal estos valores son restaurados y la rutina finaliza mediante RETI.

Para evitar mantener la rutina de interrupción ocupada durante la transmisión de una cadena completa mediante USART, se utilizó además una bandera denominada AVISO. Cuando se detecta el obstáculo, la interrupción únicamente activa esta bandera. Posteriormente, el programa principal reconoce el aviso y realiza la transmisión de los mensajes correspondientes. De esta forma se prioriza la detención del movimiento y se mantiene la rutina de interrupción lo más corta posible.

3) *Comunicación USART*: La comunicación serial se configuró para trabajar a 9600 baudios, utilizando 8 bits de datos, sin paridad y un bit de parada. Los registros UBRR0H y UBRR0L fueron utilizados para establecer la velocidad de comunicación, mientras que UCSRB y UCSRC permitieron habilitar el transmisor y configurar el formato de la trama.

En esta aplicación solamente fue necesario utilizar la transmisión, debido a que USART se emplea para informar el estado del sistema y no para recibir órdenes de control. Los caracteres son enviados utilizando el registro UDR0.

Los diferentes mensajes fueron almacenados en la memoria de programa. Para recorrer cada cadena se utilizó el registro Z junto con la instrucción LPM. Cada mensaje termina con un byte de valor cero, utilizado por la rutina para detectar el final del texto.

Durante el funcionamiento normal se transmiten los mensajes “Puerta abriendo”, “Puerta abierta”, “Puerta cerrando” y “Puerta cerrada”. Si actúa el sistema de seguridad también se transmiten los mensajes “Obstaculo detectado” y “Movimiento detenido por seguridad”.

4) *Simulación en PICSIMLAB*: Una vez finalizado el programa se realizó la compilación en Microchip Studio y se verificó que no presentara errores. El archivo .hex generado fue cargado posteriormente en PICSIMLAB utilizando una placa Arduino Uno virtual.

Para representar las entradas se utilizaron cinco pulsadores: dos para las órdenes de apertura y cierre, dos para representar los sensores de puerta completamente abierta y completamente cerrada, y uno para representar el sensor de obstáculos.

Las salidas PB1 y PB2 permitieron comprobar las órdenes correspondientes a los dos sentidos de movimiento. La salida PB3 fue utilizada para la alarma. También se agregó un terminal virtual conectado al transmisor USART del microcontrolador, permitiendo observar los mensajes enviados durante cada cambio de estado.

Durante las pruebas se verificó inicialmente el ciclo normal de apertura y cierre. Posteriormente se activó el sensor de obstáculos mientras el sistema se encontraba en movimiento, con el objetivo de comprobar la respuesta de la interrupción de seguridad.

5) *Versionado del programa:* El desarrollo del programa fue gestionado mediante Git y GitHub, incorporando las diferentes funciones de forma progresiva. Entre las principales modificaciones registradas se encuentran la implementación inicial de la máquina de estados y las entradas y salidas, la incorporación de la interrupción de seguridad mediante PCINT0 y posteriormente la comunicación USART para informar los diferentes estados del sistema.

El código completo y el historial de modificaciones realizadas se encuentran disponibles en el repositorio público indicado en el Apéndice.

6) *Implementación física:* Para la implementación física se planteó utilizar cinco pulsadores para representar las dos órdenes del control remoto, los dos sensores de posición y el sensor de obstáculos. Debido a que no se dispone de una etapa de potencia para realizar la inversión de giro de un motor DC, las órdenes de apertura y cierre son representadas mediante dos LEDs conectados a PB1 y PB2.

De esta manera es posible comprobar la lógica de control de ambos sentidos sin conectar directamente una carga de potencia a las salidas del microcontrolador. La alarma es representada mediante un buzzer conectado a PB3. La implementación mantiene la misma distribución de entradas y salidas utilizada previamente durante la simulación.

VII. RESULTADOS Y DISCUSIÓN

A. Control de matriz de LEDs

Se logró implementar correctamente el control de una matriz LED de 8×8 mediante el microcontrolador ATmega328P, utilizando comunicación USART para seleccionar las diferentes opciones de visualización. El funcionamiento del sistema fue comprobado tanto mediante simulación en Proteus como a través del montaje físico.

Durante las pruebas se verificó que, al enviar los caracteres correspondientes desde el monitor serial, la matriz respondiera de acuerdo con lo programado. Se consiguió visualizar un mensaje desplazable y dos dibujos diferentes, comprobando así el correcto funcionamiento de la comunicación serial y de la multiplexación utilizada para controlar las filas y columnas de la matriz.

Durante la implementación física fue necesario realizar algunos ajustes respecto a la lógica utilizada inicialmente para controlar la matriz. En las primeras pruebas los LED permanecían encendidos cuando deberían encontrarse apagados, por lo que se modificaron los estados lógicos de activación de las filas y columnas. Una vez realizado este ajuste, fue posible controlar correctamente qué LED debían permanecer encendidos o apagados.

También se realizaron ajustes en la representación de los patrones para obtener la orientación correcta de los dibujos en la matriz. Luego de estas modificaciones, el comportamiento del montaje físico fue consistente con el obtenido en la simulación realizada en Proteus.

En general, los resultados obtenidos fueron satisfactorios, ya que se logró cumplir con el objetivo de controlar una matriz LED mediante el ATmega328P, implementar la comunicación

USART y representar correctamente diferentes patrones y un mensaje desplazable. La tarea permitió además comprobar de forma práctica la importancia de la configuración de los puertos, la lógica de activación de la matriz y la multiplexación para el control de dispositivos LED.

B. Conversor DAC R-2R

Las pruebas realizadas sobre la implementación física permitieron comprobar el funcionamiento del sistema de generación de señales mediante LUT y DAC R-2R.

La comunicación USART funcionó correctamente a 9600 baudios. Al iniciar el sistema se mostró el menú en el monitor serial y fue posible seleccionar las señales 11 y 3 mediante los caracteres correspondientes. Los comandos '+' y '-' permitieron modificar la velocidad de generación, mientras que 'M' volvió a mostrar las opciones del menú.

La salida del DAC R-2R fue observada mediante un osciloscopio. Al seleccionar la señal 3 se obtuvo una forma de onda periódica tipo rampa, correspondiente al incremento progresivo de las muestras desde 0x00 hasta 0xFF. Para la señal 11 se observó una forma aproximadamente triangular, producida por el aumento y posterior disminución de los valores almacenados en la LUT.

También se probaron los tres modos de velocidad programados. En el modo lento se observó un período mayor, mientras que en los modos medio y rápido la señal se repitió con mayor frecuencia. En los tres casos la forma general de la señal se mantuvo, por lo que el cambio de OCR1A permitió modificar principalmente el tiempo entre muestras sin cambiar los valores de la LUT.

Estos resultados permitieron comprobar el funcionamiento conjunto de las tablas LUT, el Timer1, la comunicación USART y la red R-2R utilizada como conversor digital-analógico. Las dos señales asignadas pudieron generarse correctamente y su frecuencia pudo modificarse durante la ejecución.

La simulación realizada en Proteus presentó el mismo comportamiento general que la implementación física, permitiendo visualizar ambas señales y comprobar el cambio de frecuencia mediante los comandos del menú serial.

C. Control del plotter

Se comprobó el funcionamiento de la comunicación serial asíncrona USART configurada a 9600 baudios ($UBRR0 = 103$), operando con una trama de 8 bits de datos, sin paridad y 1 bit de parada. Al energizar el microcontrolador o reiniciar el sistema, la rutina de transmisión mediante lectura de memoria Flash (LPM) desplegó en la terminal serie la interfaz de usuario sin errores de trama:

```
PLOTTER UTEC
1 - Triangulo
2 - Circulo
3 - Pentagrama
4 - Figura libre
P - Gengar
T - Todas
Seleccione:
```

Se evaluó la recepción de comandos tanto en mayúsculas como en minúsculas (ej., 'P' y 'p'), confirmando que el microcontrolador ATmega328P decodifica la opción e inicia inmediatamente la secuencia de dibujo correspondiente.

1) *Evaluación de Trazado y Fidelidad Geométrica:* La validación física de las figuras trazadas se llevó a cabo sobre la superficie de trabajo del plotter accionando el lápiz mediante el solenoide neumático. Se evaluaron experimentalmente todas las opciones del menú:

1) **Figuras Geométricas Básicas (Opciones 1, 2 y 3):**

- **Triángulo (opcion_1):** Cierre preciso en el punto de origen mediante la combinación de 60 pasos diagonales y 120 pasos horizontales en la base.
- **Círculo (opcion_2):** Interpolación de 12 arcos continuos alternando pasos ortogonales y diagonales suaves, logrando una circunferencia uniforme sin vértices rectos marcados.
- **Pentagrama (opcion_3):** Trazo de una estrella simétrica de 5 puntas compuesta por tramos uniformes de 120 pasos por lado.

2) **Figura Libre: Corazón (opcion_4):** Trazado simétrico ejecutado en 6 etapas continuas mediante trayectorias diagonales. El control neumático garantizó que el lápiz descendiera únicamente durante el contorno.

3) **Pokémon Asignado: Gengar (opcion_P):**

- **Silueta Exterior:** La lectura secuencial de la Look-Up Table (GENGAR_MATRIZ) en memoria Flash decodificó correctamente los vectores de dirección. La compensación en software para la inversión física del eje X permitió dibujar la silueta en la orientación correcta.
- **Detalles Internos:** Tras completar la silueta, el sistema levantó el lápiz (lapis_arriba), se posicionó en coordenadas absolutas y ejecutó con precisión los ojos y la boca con sus 3 dientes verticales.

4) **Secuencia Completa (opcion_T):** Se ejecutaron consecutivamente las 5 figuras intercalando desplazamientos con el lápiz arriba (pasos_derecha y pasos_abajo), logrando distribuir los dibujos en la hoja dentro de los límites programados en el PLC sin colisiones ni superposiciones[cite: 1].

D. Comunicación USART entre microcontroladores

La validación del sistema se llevó a cabo evaluando la correspondencia entre los estados lógicos de entrada en el emisor y la activación del LED respectivo en el receptor.

1) *Verificación de la Tabla de Verdad Decodificada:* La Tabla III sintetiza las pruebas realizadas sobre el conjunto de combinaciones binarias posibles. Se verificó el correcto procesamiento del código en cada estado de la máquina.

2) *Análisis de Funcionamiento:*

- **Sincronización y Tasa de Error:** La comunicación USART a 9600 bps garantizó la transmisión limpia de los marcos sin requerir tiempos de retardo (delays) adicionales en el bucle principal. El sondeo directo de

TABLE III
TABLA DE VERDAD Y RESPUESTAS REGISTRADAS EN EL RECEPTOR

Entrada (PD4,PD3,PD2)	Binario	Decimal	Salida Activa	Puerto/Pin Físico	Estado Registros (PORTC, PORTB)
000	00000000b	0	LED 0	PORTC0 (Pin A0)	PORTC = 0x01, PORTB = 0x00
001	00000001b	1	LED 1	PORTC1 (Pin A1)	PORTC = 0x02, PORTB = 0x00
010	00000010b	2	LED 2	PORTC2 (Pin A2)	PORTC = 0x04, PORTB = 0x00
011	00000011b	3	LED 3	PORTC3 (Pin A3)	PORTC = 0x08, PORTB = 0x00
100	00000100b	4	LED 4	PORTC4 (Pin A4)	PORTC = 0x10, PORTB = 0x00
101	00000101b	5	LED 5	PORTC5 (Pin A5)	PORTC = 0x20, PORTB = 0x00
110	00000110b	6	LED 6	PORTB0 (Pin D8)	PORTC = 0x00, PORTB = 0x01
111	00000111b	7	LED 7	PORTB1 (Pin D9)	PORTC = 0x00, PORTB = 0x02

las banderas UDRE0 y RXC0 evitó la pérdida de tramas o sobrescritura en el *buffer*.

- **Commutación Exclusiva de Puertos:** Se constató experimentalmente que el reseteo explícito de los puertos no seleccionados (clr r19 aplicado a PORTB o PORTC) eliminó problemas de encendidos solapados al transicionar entre el grupo de salidas 0–5 y el grupo 6–7.
- **Integridad del Acondicionamiento de Datos:** Las operaciones de desplazamiento a la derecha (lsr) en el emisor permitieron desacoplar la ubicación física de los pulsadores respecto a la representación del dato transmitido, haciendo eficiente el uso del ancho de banda serie (1 byte transmitido por muestra).

E. Sistema de control de puerta de garaje

Las pruebas realizadas en PICSimLab permitieron comprobar el funcionamiento de la máquina de estados implementada para la puerta de garaje. Al iniciar el programa, el sistema permanece en el estado CERRADA, manteniendo desactivadas las dos órdenes de movimiento y la alarma. Mediante USART también se transmite el mensaje correspondiente al estado inicial.

Al activar el pulsador de apertura, el sistema cambia al estado ABRIENDO. En esta condición se activa la salida PB1 y la alarma, mientras que mediante USART se transmite el mensaje “Puerta abriendo”. El sistema permanece en este estado hasta que se activa el sensor correspondiente a la posición completamente abierta. En ese momento el movimiento se detiene, la alarma se desactiva y el sistema cambia al estado ABIERTA, transmitiéndose también el mensaje correspondiente.

El procedimiento de cierre presentó un comportamiento equivalente. Al activar la orden de cierre desde el estado ABIERTA, el sistema pasa a CERRANDO, activando la salida PB2 y la alarma. Cuando se detecta el sensor de puerta completamente cerrada, las salidas son desactivadas y el sistema regresa al estado CERRADA.

También se comprobó el funcionamiento de la interrupción de seguridad. Al activar el sensor de obstáculos durante cualquiera de los dos estados de movimiento, las salidas correspondientes al motor y la alarma fueron desactivadas y el sistema pasó al estado SEGURIDAD. Posteriormente se visualizaron en el terminal USART los mensajes “Obstaculo detectado” y “Movimiento detenido por seguridad”.

Una vez retirado el obstáculo, el sistema permanece detenido hasta detectar una nueva orden del usuario. Al recibir una nueva orden de apertura o cierre se abandona el estado SEGURIDAD y se continúa el funcionamiento en el sentido seleccionado.

Una de las partes que requirió mayor cuidado durante el desarrollo fue la incorporación del sensor de obstáculos mediante una interrupción diferente de INT0 e INT1. La utilización de PCINT0 permitió cumplir esta condición, aunque fue necesario considerar que una interrupción por cambio de pin puede generarse tanto al activar como al liberar la entrada. Por esta razón, dentro de la rutina se vuelve a comprobar el estado de PB0 antes de realizar la detención.

Otra decisión tomada durante el desarrollo fue evitar la transmisión de los mensajes USART directamente dentro de la rutina de interrupción. En su lugar se utilizó una bandera que indica al programa principal que existe un aviso pendiente. Esto permitió mantener corta la rutina de seguridad y realizar primero la acción más importante, que consiste en detener las salidas asociadas al movimiento y a la alarma.

En general, la simulación permitió verificar correctamente los estados de apertura y cierre, la lectura de los sensores, la actuación de la alarma, la respuesta ante un obstáculo y la transmisión de los mensajes mediante USART.

La implementación física presentó el mismo comportamiento general observado durante la simulación. Los dos LEDs utilizados para representar los sentidos de movimiento permitieron verificar las órdenes de apertura y cierre, mientras que el buzzer indicó correctamente los estados de movimiento. Los cinco pulsadores respondieron según las funciones asignadas y el sensor de obstáculos permitió comprobar nuevamente la detención mediante la interrupción de seguridad.

VIII. CONCLUSIONES

- Se desarrollaron e integraron con éxito los distintos sistemas en el microcontrolador ATmega328P utilizando lenguaje ensamblador, logrando un control preciso y un procesamiento eficiente de recursos en tiempo real.
- Se validó el funcionamiento del control de la matriz de LEDs, la conversión digital-analógica mediante el DAC R-2R y el movimiento del plotter a través de una adecuada gestión de las salidas del microcontrolador.
- El uso de Timers, interrupciones, tablas LUT y la comunicación USART permitió sincronizar las tareas del sistema de manera determinista, optimizando el tiempo de ejecución y garantizando la fluidez en el procesamiento de datos.

IX. APÉNDICE

Nota: Como material complementario al presente informe se incluyen los códigos desarrollados, simulaciones, fotografías y registros audiovisuales correspondientes a las diferentes actividades realizadas durante el laboratorio.

El material complementario se encuentra organizado en los siguientes repositorios digitales:

- **Repositorio de GitHub:** contiene los códigos fuente desarrollados en lenguaje ensamblador para el ATmega328P, junto con el historial de modificaciones realizadas durante el desarrollo del laboratorio.

- **Repositorio de Google Drive:** contiene las evidencias complementarias de las actividades realizadas, incluyendo capturas de las simulaciones, fotografías de los montajes y registros audiovisuales del funcionamiento de los sistemas implementados.

El código fuente del proyecto puede consultarse en:

Repositorio de GitHub (clic aquí)

Las evidencias y el material audiovisual pueden consultarse en:

Repositorio de Google Drive (clic aquí)

REFERENCES

- [1] A. Herrera, "Multiplexación de una matriz de LEDs," *Avelino Herrera Blog*. [En línea]. Disponible en: <https://avelinoherrera.com/blog/index.php?m=11&y=13&entry=entry131112-112351>. [Accedido: 21-sep-2026].
- [2] Instructables, "Multiplexing with Arduino and the 74HC595." [En línea]. Disponible en: <https://www.instructables.com/Multiplexing-with-Arduino-and-the-74HC595/>. [Accedido: 21-sep-2026].
- [3] Wikipedia, "Universal asynchronous receiver-transmitter." [En línea]. Disponible en: https://en.wikipedia.org/wiki/Universal_asynchronous_receiver-transmitter. [Accedido: 21-sep-2026].
- [4] Microgamma, "Tablas LUT y LUT 3D." [En línea]. Disponible en: <https://www.microgamma.com/tecnologia/tablas-lut-i-lut-3d.php>. [Accedido: 21-sep-2026].
- [5] Roon Labs Community, "What exactly is an R2R DAC?" [En línea]. Disponible en: <https://community.roonlabs.com/t/what-exactly-is-an-r2r-dac/22735>. [Accedido: 21-sep-2026].
- [6] Electronics Tutorials, "R-2R Digital-to-Analogue Converter (DAC)." [En línea]. Disponible en: <https://www.electronics-tutorials.ws/combination/r-2r-dac.html>. [Accedido: 21-sep-2026].
- [7] RoboticLab, "AVR Timers." [En línea]. Disponible en: <https://home.roboticlab.eu/en/avr/timers>. [Accedido: 21-sep-2026].
- [8] Control Automático Educación, "Interrupciones en microcontroladores." [En línea]. Disponible en: <https://controlautomaticoeducacion.com/sistemas-embebidos/microcontroladores-pic/interrupciones/>. [Accedido: 21-sep-2026].
- [9] Quora, "What is a stack pointer in a microprocessor?" [En línea]. Disponible en: <https://www.quora.com/What-is-a-stack-pointer-in-a-microprocessor>. [Accedido: 21-sep-2026].
- [10] MAPA, "Equipos plotter." [En línea]. Disponible en: <https://www.mapa.com.uy/detalle-equipos.php?id=13>. [Accedido: 21-sep-2026].
- [11] Instructables, "Plotter – Manufactura Digital." [En línea]. Disponible en: <https://www.instructables.com/Plotter-Manufactura-Digital/>. [Accedido: 21-sep-2026].
- [12] Industrial Monitor Direct, "PLC output types: Relay vs. Triac vs. MOSFET comparison." [En línea]. Disponible en: <https://industrialmonitordirect.com/es/blogs/knowledgebase/plc-output-types-relay-vs-triac-vs-mosfet-comparison>. [Accedido: 21-sep-2026].